

**COMPLETE OPERATIONS GUIDE
YUGABYTEDB 3-NODE CLUSTER**

**Installation, Secure Access, Sample Schema & Port
Management**

CentOS 8+ (RHEL-compatible) • 3-Node Cluster • Replication Factor 3

Prepared for: [Mukesh Sharma](#)

September 2026

Table of Contents

1. Overview and Cluster Topology

This guide covers the complete lifecycle for a secure, production-oriented 3-node YugabyteDB cluster on CentOS 8 (or any RHEL 8/9-compatible distribution): OS preparation, installation, systemd service configuration, port policy (what must be open, what must be blocked), password-protected command-line access (YSQL and YCQL), a password-protected Admin Web UI via an nginx reverse proxy, and loading a sample schema to validate the cluster end to end.

	node1	node2	node3
IP address	192.168.87.133	192.168.87.194	192.168.87.195
Roles	yb-master + yb-tserver + nginx	yb-master + yb-tserver + nginx	yb-master + yb-tserver + nginx
OS	CentOS 8+	CentOS 8+	CentOS 8+
YugabyteDB version	2026.1.1.1	2026.1.1.1	2026.1.1.1
Replication factor	3	3	3

Substitute your own values: Every command in this guide uses these three IPs. Replace them with your environment's actual addresses throughout — the pattern (identical cluster-membership flags, node-specific bind addresses) stays the same regardless of the actual IPs used.

1.1 What ends up open vs. blocked (summary)

Full detail is in Section 3. Summary of the end state this guide builds toward:

Port	Service	Reachable from	Purpose
7100	yb-master	Cluster nodes only	Master Raft/RPC
9100	yb-tserver	Cluster nodes only	TServer Raft/RPC
5433	yb-tserver	App/client hosts	YSQL (PostgreSQL protocol) — password required
9042	yb-tserver	App/client hosts	YCQL (Cassandra protocol) — password required
7080	nginx	Admin/ops hosts	Master Web UI — password required (proxy)
9080	nginx	Admin/ops hosts	TServer Web UI — password required (proxy)
7000	yb-master	BLOCKED externally (localhost only)	Raw, unauthenticated admin UI — never expose directly
9000	yb-tserver	BLOCKED externally (localhost only)	Raw, unauthenticated admin UI — never expose directly

Port	Service	Reachable from	Purpose
12000 / 13000	yb-tserver	Monitoring hosts only	YCQL/YSQL Prometheus metrics — restrict, don't fully open

2. Prerequisites (Run on All 3 Nodes)

Note: CentOS 8 reached end-of-life in December 2021 (no further OS security patches). These steps work as-is on CentOS 8, and identically on CentOS Stream 8/9, Rocky Linux 8/9, or AlmaLinux 8/9.

2.1 Hostnames and /etc/hosts

If nodes were cloned from a shared VM template, verify each has a unique hostname first — cloned VMs commonly retain the template's original hostname.

```
# On each node individually, matching its own IP:
sudo hostnamectl set-hostname node1      # on 192.168.87.133
sudo hostnamectl set-hostname node2      # on 192.168.87.194
sudo hostnamectl set-hostname node3      # on 192.168.87.195

# Push this identical block to ALL three nodes:
sudo tee -a /etc/hosts <<'EOF'
192.168.87.133 node1
192.168.87.194 node2
192.168.87.195 node3
EOF
```

Also check: Cloned VMs can duplicate /etc/machine-id. Run `cat /etc/machine-id` on all three — if identical, regenerate on two of them with `sudo rm /etc/machine-id && sudo systemd-machine-id-setup`.

2.2 Base packages

```
sudo yum install -y wget which chrony python3 tar policycoreutils-python-utils httpd-tools
nginx
sudo ln -sf /usr/bin/python3 /usr/bin/python
```

2.3 Time synchronization

```
sudo systemctl enable --now chronyd
sudo tee -a /etc/chrony.conf <<'EOF'
minpoll 4
maxpoll 4
EOF
sudo systemctl restart chronyd
chronyc tracking      # confirm "Leap status: Normal"
```

2.4 Kernel and ulimit tuning

```
sudo tee /etc/security/limits.d/99-yugabyte.conf <<'EOF'
*                -      core                unlimited
*                -      data                unlimited
*                -      fsize               unlimited
*                -      sigpending          119481
*                -      memlock             64
*                -      rss                 unlimited
*                -      nofile              1048576
*                -      msgqueue            819200
*                -      stack               8192
```

```

*           -      cpu          unlimited
*           -      nproc        12000
*           -      locks        unlimited
EOF

sudo tee /etc/sysctl.d/99-yugabyte.conf <<'EOF'
vm.swappiness = 0
vm.max_map_count = 262144
EOF
sudo sysctl --system

sudo tee /etc/systemd/system/disable-thp.service <<'EOF'
[Unit]
Description=Set Transparent Huge Pages to madvise
[Service]
Type=oneshot
ExecStart=/bin/sh -c "echo madvise > /sys/kernel/mm/transparent_hugepage/enabled; echo
madvise > /sys/kernel/mm/transparent_hugepage/defrag"
[Install]
WantedBy=multi-user.target
EOF
sudo systemctl enable --now disable-thp

```

Log out and back in (or reboot) so the new ulimits apply to new shells and services.

2.5 Service account and data directories

```

sudo useradd -m -s /bin/bash yugabyte
sudo mkdir -p /opt/yugabyte /opt/data0/yb-data /opt/data1/yb-data
sudo chown -R yugabyte:yugabyte /opt/yugabyte /opt/data0 /opt/data1

```

3. Firewall and Port Policy (Run on All 3 Nodes)

Get this right once, up front — the single most common source of confusing failures in a distributed database deployment is a firewall that silently drops traffic (looks like a hang or timeout) rather than actively refusing it (which at least fails fast and obviously). This section opens exactly what's needed between cluster nodes and clients, and explicitly blocks the two ports that must never be reachable from outside the host.

3.1 Ports required between cluster nodes

yb-master and yb-tserver on each node must reach the same ports on the other two nodes for Raft consensus, leader election, and data replication:

```
sudo firewall-cmd --permanent --zone=public --add-port=7100/tcp # yb-master RPC
sudo firewall-cmd --permanent --zone=public --add-port=9100/tcp # yb-tserver RPC
```

3.2 Ports required for client/application access

```
sudo firewall-cmd --permanent --zone=public --add-port=5433/tcp # YSQL (PostgreSQL
protocol)
sudo firewall-cmd --permanent --zone=public --add-port=9042/tcp # YCQL (Cassandra
protocol)
```

3.3 Ports required for the password-protected Admin Web UI (nginx)

```
sudo firewall-cmd --permanent --zone=public --add-port=7080/tcp # Master UI, via nginx
(password-protected)
sudo firewall-cmd --permanent --zone=public --add-port=9080/tcp # TServer UI, via nginx
(password-protected)
```

Easy to miss: 7080/9080 are proxy ports, not YugabyteDB's own — they don't exist until nginx is configured in Section 8. If you open them here but skip Section 8, connections will time out or refuse until nginx is actually listening.

3.4 Optional: metrics endpoints (restrict rather than fully open)

Ports 12000 (YCQL metrics) and 13000 (YSQL metrics) expose Prometheus-format metrics with no authentication. If you run Prometheus/Grafana, restrict these to your monitoring host(s) specifically rather than opening them broadly:

```
sudo firewall-cmd --permanent --zone=public --add-rich-rule='rule family="ipv4" source
address="<monitoring-host-ip>" port port="12000" protocol="tcp" accept'
sudo firewall-cmd --permanent --zone=public --add-rich-rule='rule family="ipv4" source
address="<monitoring-host-ip>" port port="13000" protocol="tcp" accept'
```

Skip this entirely if you have no metrics collector yet — do not open 12000/13000 broadly "just in case."

3.5 Block direct access to the raw, unauthenticated admin ports (7000, 9000)

yb-master's and yb-tserver's native web servers listen on 7000 and 9000 with no authentication of their own — Section 8's nginx proxy is what adds a password, on different ports (7080/9080). 7000 and 9000 themselves must never be reachable from outside the local host:

```
sudo firewall-cmd --permanent --zone=public --add-rich-rule='rule family="ipv4" port
port="7000" protocol="tcp" reject'
sudo firewall-cmd --permanent --zone=public --add-rich-rule='rule family="ipv4" port
port="9000" protocol="tcp" reject'
```

This works because nginx's proxy connection to these ports originates from the same host (delivered over the loopback interface, in firewalld's trusted zone), while genuine external connections hit this reject rule. Verify both properties after Section 8's nginx setup is complete: a curl to these ports from a different node should fail, while nginx's own local proxying to them should keep working.

3.6 Apply and verify

```
sudo firewall-cmd --reload
sudo firewall-cmd --zone=public --list-all
```

Confirm the output shows 7100, 9100, 5433, 9042, 7080, and 9080 under ports:, and the two reject rich-rules for 7000/9000 listed separately. Repeat this entire Section 3 identically on all three nodes.

4. Download and Install YugabyteDB Software (All 3 Nodes)

Run as the **yugabyte** service user on every node. Check download.yugabyte.com for the current stable version if newer than the one below.

```
sudo su - yugabyte
cd /opt/yugabyte

VERSION=2026.1.1.1
wget https://software.yugabyte.com/releases/${VERSION}/yugabyte-${VERSION}-linux-x86_64.tar.gz
tar xvfz yugabyte-${VERSION}-linux-x86_64.tar.gz

# Confirm the ACTUAL extracted directory name before writing any systemd unit -
# the tarball filename may include a build suffix (e.g. "-b2") that the extracted
# directory does not carry. Using the wrong path here is the #1 cause of a
# systemd "status=203/EXEC" failure later.
ls /opt/yugabyte/

cd yugabyte-${VERSION}
./bin/post_install.sh
```

4.1 Confirm the binary is present and executable

```
ls -l /opt/yugabyte/yugabyte-2026.1.1.1/bin/yb-master
sudo -u yugabyte /opt/yugabyte/yugabyte-2026.1.1.1/bin/yb-master --version
```

5. Configure yb-master and yb-tserver as systemd Services

These unit files include YSQL and YCQL password-authentication flags from the start (`--ysql_enable_auth`, `--use_cassandra_authentication`), so the cluster comes up already requiring passwords rather than needing a later retrofit.

Write ExecStart as a single line. A multi-line ExecStart using trailing backslash (\) continuations is fragile: forgetting a single backslash silently truncates the command, and systemd logs only a terse "Unknown lvalue" warning rather than failing loudly. Every ExecStart in this guide is deliberately written as one line for exactly this reason.

5.1 node1 (192.168.87.133)

/etc/systemd/system/yb-master.service:

```
sudo tee /etc/systemd/system/yb-master.service <<'EOF'
[Unit]
Description=YugabyteDB Master
After=network.target

[Service]
User=yugabyte
Group=yugabyte
ExecStart=/opt/yugabyte/yugabyte-2026.1.1.1/bin/yb-master --master_addresses
192.168.87.133:7100,192.168.87.194:7100,192.168.87.195:7100 --rpc_bind_addresses
192.168.87.133:7100 --fs_data_dirs "/opt/data0/yb-data,/opt/data1/yb-data" --
replication_factor=3
Restart=always
LimitNOFILE=1048576
LimitNPROC=12000

[Install]
WantedBy=multi-user.target
EOF
```

/etc/systemd/system/yb-tserver.service:

```
sudo tee /etc/systemd/system/yb-tserver.service <<'EOF'
[Unit]
Description=YugabyteDB TServer
After=network.target

[Service]
User=yugabyte
Group=yugabyte
ExecStart=/opt/yugabyte/yugabyte-2026.1.1.1/bin/yb-tserver --tserver_master_addr
192.168.87.133:7100,192.168.87.194:7100,192.168.87.195:7100 --rpc_bind_addresses
192.168.87.133:9100 --enable_ysql --pgsql_proxy_bind_address 192.168.87.133:5433 --
cql_proxy_bind_address 192.168.87.133:9042 --fs_data_dirs "/opt/data0/yb-data,/opt/data1/yb-
data" --ysql_enable_auth=true --use_cassandra_authentication=true
Restart=always
LimitNOFILE=1048576
LimitNPROC=12000

[Install]
```

```
WantedBy=multi-user.target
EOF
```

5.2 node2 (192.168.87.194) and node3 (192.168.87.195)

Identical files, changing only the three per-node IP occurrences:

Flag	node2 value	node3 value
--rpc_bind_addresses (master)	192.168.87.194:7100	192.168.87.195:7100
--rpc_bind_addresses (tserver)	192.168.87.194:9100	192.168.87.195:9100
--pgsql_proxy_bind_address	192.168.87.194:5433	192.168.87.195:5433
--cql_proxy_bind_address	192.168.87.194:9042	192.168.87.195:9042

--master_addresses / --tserver_master_addrs, --fs_data_dirs, --replication_factor, --ysql_enable_auth, and --use_cassandra_authentication stay byte-for-byte identical to node1's files on both node2 and node3.

Do not bind to another node's IP. The single most common mistake when copying these files between nodes is forgetting to update all three IP occurrences, leaving one node's service bound to a different node's address. This breaks Raft/Paxos quorum formation while systemd still happily reports the service as "active." Double check with ``ip a`` on each node before assuming a copy-pasted file is correct.

5.3 Enable and start on all 3 nodes

```
sudo systemctl daemon-reload
sudo systemctl enable --now yb-master
sudo systemctl enable --now yb-tserver
sudo systemctl status yb-master yb-tserver --no-pager
```

Confirm both show "active (running)" — not "activating (auto-restart)," which indicates a crash loop. If you see a crash loop, check: (1) the binary path in ExecStart matches the actual extracted directory exactly, (2) ``ps aux | grep yb-`` and ``journalctl -u yb-master -n 50`` for the specific error.

5.4 Set the YSQL and YCQL superuser passwords

The cluster is now enforcing authentication, but the default superusers (yugabyte / cassandra) still have their default passwords. Set real ones immediately — from any single node, since role metadata is shared cluster-wide:

```
cd /opt/yugabyte/yugabyte-2026.1.1.1

# YSQL — connect once with the default password 'yugabyte', then change it:
./bin/ysqlsh -h 192.168.87.133 -p 5433 -U yugabyte -W
yugabyte=# ALTER ROLE yugabyte PASSWORD 'YourStrongPasswordHere';
yugabyte=# \q

# YCQL — connect once with the default cassandra/cassandra, then change it:
./bin/ycqlsh 192.168.87.133 9042 -u cassandra -p cassandra
```

```
cassandra@ycqlsh> ALTER ROLE cassandra WITH PASSWORD = 'YourStrongPasswordHere';  
cassandra@ycqlsh> exit
```

ycqlsh syntax note: Host and port are positional arguments, not just -u/-p flags. Running ``ycqlsh -u cassandra -p cassandra`` with no host defaults to 127.0.0.1:9042 and fails with "Connection refused" since the tserver's CQL proxy is bound to the node's real IP, not loopback. Always specify: `ycqlsh <node-ip> 9042 -u ... -p ...`

6. Verify the Cluster

6.1 Command-line verification

```
cd /opt/yugabyte/yugabyte-2026.1.1.1

./bin/yb-admin --master_addresses 192.168.87.133:7100,192.168.87.194:7100,192.168.87.195:7100
list_all_masters
./bin/yb-admin --master_addresses 192.168.87.133:7100,192.168.87.194:7100,192.168.87.195:7100
list_all_tablet_servers

# Confirms all 3 nodes share one universe (requires the yugabyte password now):
./bin/ysqlsh -h 192.168.87.133 -p 5433 -U yugabyte -W -c "select * from yb_servers();"
```

A healthy result shows all three IPs under the same `universe_uuid`, each `node_type` = `primary` — confirming one correctly-formed 3-node cluster.

6.2 Confirm systemd health on every node

```
ps aux | grep -E 'yb-master|yb-tserver' | grep -v grep      # exactly one of each, owned by
yugabyte
sudo systemctl is-enabled yb-master yb-tserver              # "enabled" = survives reboot
sudo systemctl is-active  yb-master yb-tserver              # "active"
```

7. Secure Command-Line Access

Section 5.4 already enabled and set passwords for both client protocols. This section covers day-to-day authenticated connection patterns for YSQL and YCQL, plus creating least-privilege application users instead of using the superusers everywhere.

7.1 YSQL (ysqlsh / PostgreSQL-compatible)

```
cd /opt/yugabyte/yugabyte-2026.1.1.1
./bin/ysqlsh -h 192.168.87.133 -p 5433 -U yugabyte -W
Password:
```

Create a less-privileged application role instead of using the yugabyte superuser from applications:

```
yugabyte=# CREATE ROLE app_user WITH LOGIN PASSWORD 'AppUserPassword' SUPERUSER=false
CREATEDB=false;
yugabyte=# GRANT ALL PRIVILEGES ON DATABASE northwind TO app_user;
```

7.2 YCQL (ycqlsh / Cassandra-compatible)

```
./bin/ycqlsh 192.168.87.133 9042 -u cassandra -p
Password:

cassandra@ycqlsh> CREATE ROLE app_user WITH PASSWORD = 'AppUserPassword' AND LOGIN = true;
```

Command history hygiene: Typing `-p <password>` directly on the command line leaves it in `~/.bash_history` in plaintext. Prefer omitting the password argument and letting the client prompt interactively — both `ysqlsh (-W)` and `ycqlsh (-p with no value)` support this.

7.3 Connecting with a standard PostgreSQL driver/ORM

Because YSQL speaks the real PostgreSQL wire protocol, any standard PostgreSQL client, driver, or ORM connects without modification — point it at port 5433 on any node.

```
# JDBC (Java)
jdbc:postgresql://192.168.87.133:5433/yugabyte?user=yugabyte&password=YourStrongPasswordHere

# psycopg2 (Python)
import psycopg2
conn = psycopg2.connect(host="192.168.87.133", port=5433, dbname="yugabyte",
                        user="yugabyte", password="YourStrongPasswordHere")

# Node.js (pg)
const client = new Client({ host: '192.168.87.133', port: 5433, database: 'yugabyte',
                             user: 'yugabyte', password: 'YourStrongPasswordHere' });
```

Multi-node connections: YugabyteDB's smart JDBC/psycopg drivers accept a comma-separated host list and load-balance new connections across all tservers automatically (`load_balance=true`). Otherwise, place a TCP load balancer (HAProxy or similar) in front of all three nodes' port 5433/9042 for production traffic rather than pointing every application at one node's IP.

7.4 Connecting from a GUI tool (pgAdmin, DBeaver, DataGrip, TablePlus)

Because YSQL is PostgreSQL wire-protocol compatible, any PostgreSQL GUI client — pgAdmin included — connects to YugabyteDB exactly as it would to a regular PostgreSQL server, with one difference: the port.

pgAdmin connection settings (Register → Server):

Field	Value
Host name/address	Any node's IP, e.g. 192.168.87.133
Port	5433 (NOT the default PostgreSQL port 5432)
Maintenance database	yugabyte (or northwind, after Section 9)
Username	yugabyte (or the app_user role from Section 7.1)
Password	As set in Section 5.4 / 7.1

Single node, no automatic failover: pgAdmin connects to whichever node IP you give it and has no awareness of the other two — if that node goes down, repoint pgAdmin at a different node's IP manually. YugabyteDB's smart JDBC/psycopg drivers (Section 7.3) handle multi-node failover automatically; a plain GUI client like pgAdmin does not.

Other PostgreSQL GUI clients (DBeaver, DataGrip, TablePlus, Beekeeper Studio, etc.) use the same four settings — host, port 5433, database, and the YSQL username/password. Standard table browsing, the query tool, and schema navigation all work normally; YugabyteDB-specific internals (tablet catalogs, the `yb_servers()` function) may render slightly differently than on vanilla PostgreSQL, but this doesn't affect day-to-day use.

8. Securing Web UI Access (nginx Reverse Proxy)

YugabyteDB's native admin web servers (port 7000 for yb-master, 9000 for yb-tserver) have no authentication of their own. YugabyteDB does provide native `--webserver_password_file / --webserver_authentication_domain` flags, but in the version used for this guide (2026.1.1.1) these reliably crash yb-master/yb-tserver on startup with a generic "Webserver: Could not start on address ..." fatal error, regardless of htpasswd vs. htdigest file format. The approach in this guide — a small nginx reverse proxy handling HTTP Basic authentication in front of the native admin ports — is reliable, simple to operate, and the pattern used in most production YugabyteDB deployments regardless of whether the native flags work in a given version.

8.1 Install nginx and httpd-tools (already done in Section 2.2)

```
rpm -q nginx httpd-tools || sudo yum install -y nginx httpd-tools
```

8.2 Generate Basic-auth credentials (once, then copy to all nodes)

```
htpasswd -c /etc/nginx/yb-webui.htpasswd admin
# prompts for and confirms a password

# Copy the SAME file to the other two nodes:
scp /etc/nginx/yb-webui.htpasswd 192.168.87.194:/etc/nginx/yb-webui.htpasswd
scp /etc/nginx/yb-webui.htpasswd 192.168.87.195:/etc/nginx/yb-webui.htpasswd
```

Use `-c` only this first time (it creates the file). To add a second admin user later on any node: `htpasswd /etc/nginx/yb-webui.htpasswd anotheruser` (no `-c`, since `-c` overwrites).

8.3 nginx reverse-proxy configuration (per node)

The YugabyteDB web servers bind to the node's real IP by default (`--webserver_interface` is not set), not to 127.0.0.1 — `proxy_pass` must target that same node's own address.

On node1 (192.168.87.133):

```
sudo tee /etc/nginx/conf.d/yb-master-ui.conf <<'EOF'
server {
    listen 7080;
    location / {
        auth_basic "YugabyteDB Admin";
        auth_basic_user_file /etc/nginx/yb-webui.htpasswd;
        proxy_pass http://192.168.87.133:7000;
    }
}
EOF

sudo tee /etc/nginx/conf.d/yb-tserver-ui.conf <<'EOF'
server {
    listen 9080;
    location / {
        auth_basic "YugabyteDB Admin";
        auth_basic_user_file /etc/nginx/yb-webui.htpasswd;
```

```

    proxy_pass http://192.168.87.133:9000;
  }
}
EOF

```

On node2, use 192.168.87.194 in both proxy_pass lines. On node3, use 192.168.87.195. Not node1's IP — each node's nginx proxies to its own local YugabyteDB processes.

8.4 SELinux port labeling (required if SELinux is Enforcing)

SELinux restricts nginx to a predefined whitelist of bindable ports. 7080/9080 aren't on it by default, producing: nginx: [emerg] bind() to 0.0.0.0:7080 failed (13: Permission denied).

```

getenforce  # if "Permissive", this section can be skipped for now (but see Section 10)

sudo semanage port -a -t http_port_t -p tcp 7080
sudo semanage port -a -t http_port_t -p tcp 9080
# If a port is already claimed by another SELinux type (e.g. "ocsp_port_t" often owns 9080
# by default), semanage errors "ValueError: Port tcp/9080 already defined" — use -m instead:
sudo semanage port -m -t http_port_t -p tcp 9080

# Allow nginx to proxy OUT to the backend YugabyteDB ports:
sudo setsebool -P httpd_can_network_connect 1

```

8.5 Confirm firewall ports are open

Already done in Section 3.3/3.5 — 7080/9080 open, 7000/9000 rejected externally. If you skipped ahead, apply Section 3 now before continuing.

8.6 Enable, start, and verify

```

sudo systemctl enable --now nginx
sudo nginx -t
sudo systemctl status nginx --no-pager

curl -u admin:YourPassword http://127.0.0.1:7080/ -o /dev/null -s -w "%{http_code}\n" #
expect 200
curl -u admin:YourPassword http://127.0.0.1:9080/ -o /dev/null -s -w "%{http_code}\n" #
expect 200

```

Repeat Sections 8.2 (credentials copy) through 8.6 on all three nodes. Final URLs:

Node	Master UI (password-protected)	TServer UI (password-protected)
node1	http://192.168.87.133:7080	http://192.168.87.133:9080
node2	http://192.168.87.194:7080	http://192.168.87.194:9080
node3	http://192.168.87.195:7080	http://192.168.87.195:9080

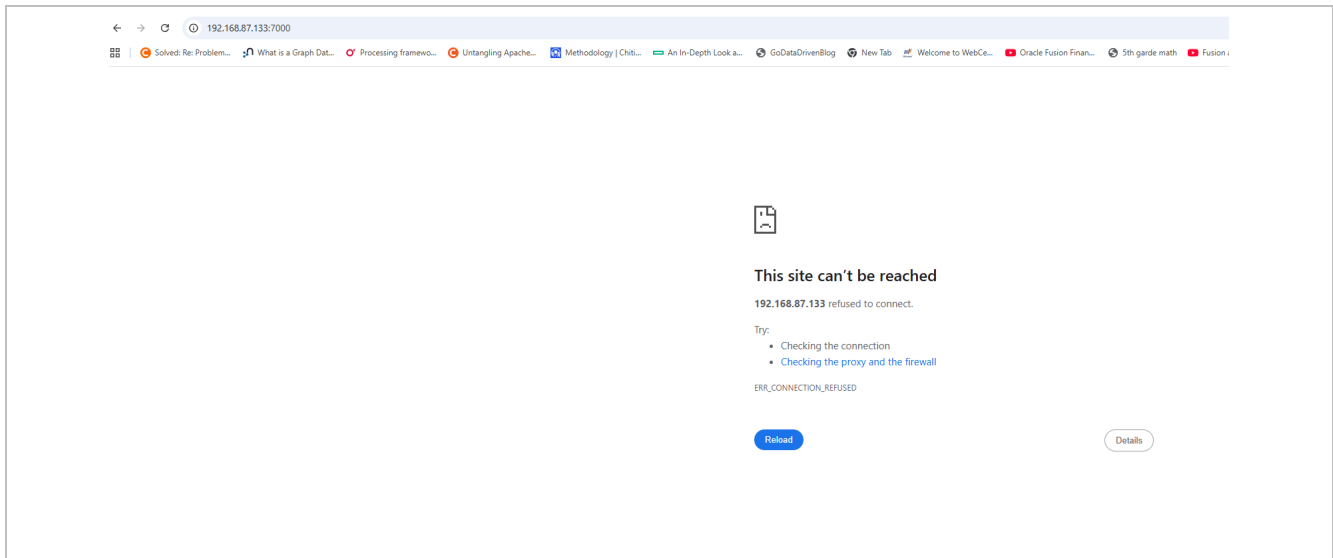
Verify the block, not just the allow: From a DIFFERENT node (or client machine), confirm the raw ports genuinely refuse direct access: ``curl -o /dev/null -s -w "%{http_code}\n" http://192.168.87.133:7000/`` should

time out or fail, while ``curl -u admin:pass -o /dev/null -s -w "%{http_code}\n" http://192.168.87.133:7080/`` should return 200. Testing only the "allow" side and assuming the "block" side works is how security gaps slip through.

Not yet encrypted: This is HTTP Basic auth over plain HTTP — credentials travel unencrypted on the wire. Acceptable on a private/trusted network; if this UI is ever reachable from anywhere less trusted, add a TLS certificate to the nginx listen blocks (listen 7080 ssl; plus ssl_certificate/ssl_certificate_key directives) before relying on it.

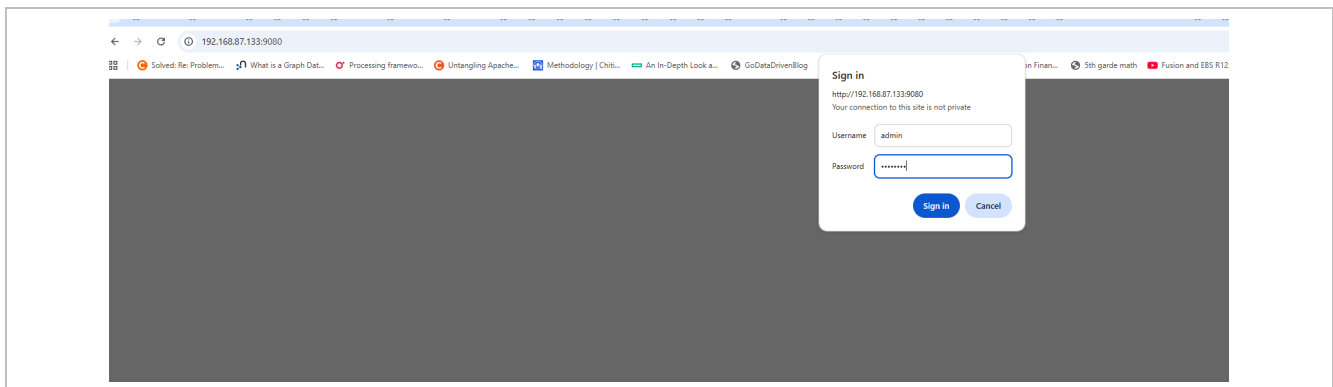
8.7 What it looks like end to end

The native admin ports (7000/9000) are unreachable directly, as configured in Section 3.5:



Direct browser access to the native yb-master port (192.168.87.133:7000) is refused — the port is blocked at the firewall, per Section 3.5.

The nginx-proxied ports (7080/9080) prompt for the Basic-auth credentials configured in Section 8.2:



Browsing to the TServer UI proxy port (192.168.87.133:9080) triggers the browser's native HTTP Basic Auth sign-in dialog before any YugabyteDB page loads.

After authenticating, the real YugabyteDB admin dashboards load normally through the proxy:

YugabyteDB Overview

Universe UUID	1a8a3d8f-415a-4a5c-8c28-46277236203
Replication Factor	3
Num Nodes (TServer)	3
Num User Tables	0
Cluster Balancer Enabled	✓
Is Cluster Load Balanced?	✓
YugabyteDB Version	2025.1.1.1
Build Type	RELEASE
Encryption Status	Never enabled

Masters/RAFT Table

Server	RAFT Role	Term	Log Size	Cloud	Region	Zone	UUID
192.168.87.133:7000	FOLLOWER	1	0.00	CLOUD	us-east-1	us-east-1a	1a8a3d8f-415a-4a5c-8c28-46277236203
192.168.87.194:7000	LEADER	0.24.58	0.00	CLOUD	us-east-1	us-east-1a	1a8a3d8f-415a-4a5c-8c28-46277236203
192.168.87.195:7000	FOLLOWER	0.24.43	0.00	CLOUD	us-east-1	us-east-1a	1a8a3d8f-415a-4a5c-8c28-46277236203

Master UI (192.168.87.133:7080) after authentication — cluster overview, replication factor 3, and the Masters/RAFT table showing all three nodes.

Dashboards

Dashboard	Description
Operations	List of operations that are currently replicating
Remote Bootstraps Sessions being served	List of remote bootstrap sessions this tablet server is currently serving
Maintenance Manager	List of operations that are currently running and those that are registered
Object locks held at the server local TSLocalLockManager	Dump of all granted and awaiting object locks at the local TSLocalLockManager

Connectivity

Type	UUID	Master State	Last seen	Endpoint	Ping
MASTER	58372a1c17ba4742a490c03a460733e1	ALIVE	22.445s	192.168.87.194:7100	3.609ms
TABLET_SERVER	0f5a5641e0a3c9f6ac211596c2073	ALIVE	22.436s	192.168.87.195:9100	9.262ms
TABLET_SERVER	ae0b44e47ad24f590f9367ac78d8a	ALIVE	22.430s	192.168.87.194:9100	14.78ms

TServer UI (192.168.87.133:9080) after authentication — the Connectivity table confirms all three nodes report ALIVE.

9. Loading a Sample Schema (Northwind)

With authentication enabled, all commands below include the password flags introduced in Sections 5 and 7. Run these from any one node — YugabyteDB replicates the data cluster-wide automatically.

9.1 Download the Northwind DDL and data files

```
cd ~
curl -O https://raw.githubusercontent.com/yugabyte/yugabyte-
db/master/sample/northwind_ddl.sql
curl -O https://raw.githubusercontent.com/yugabyte/yugabyte-
db/master/sample/northwind_data.sql
ls -la northwind_ddl.sql northwind_data.sql
```

9.2 Create the northwind database

```
ysqlsh -h 192.168.87.133 -U yugabyte -W
-- at the ysqlsh prompt:
CREATE DATABASE northwind;
\q
```

9.3 Load the schema and data

```
ysqlsh -h 192.168.87.133 -U yugabyte -d northwind -W -f northwind_ddl.sql
ysqlsh -h 192.168.87.133 -U yugabyte -d northwind -W -f northwind_data.sql
```

Large file, single connection: northwind_data.sql runs as one long transactional session — on a fresh 3-node cluster this typically completes in under a minute. If it appears to hang, check `journalctl -u yb-tserver -f` on the node you connected to rather than cancelling.

9.4 Verify the import

```
ysqlsh -h 192.168.87.133 -U yugabyte -d northwind -W
-- at the prompt:
\dt
SELECT COUNT(*) FROM orders;
SELECT COUNT(*) FROM customers;
SELECT productname, unitprice FROM products ORDER BY unitprice DESC LIMIT 5;
\q
```

Expected: 13 tables (categories, customers, employees, orders, order_details, products, shippers, suppliers, territories, etc.), 830 orders, 91 customers.

9.5 Confirm the data is truly distributed

Because YugabyteDB is a distributed database, the northwind tables' tablets are spread and replicated across all 3 nodes automatically (replication factor 3, as configured in Section 5). Confirm via the Master UI:

- Open <http://192.168.87.133:7080> (Section 8) → Tables → northwind.orders → Tablets, and note that leader/follower tablets are distributed across node1/node2/node3.

- Connect from a different node (e.g. `ysqlsh -h 192.168.87.194 -U yugabyte -d northwind -W`) and confirm the same row counts — proving it's one logical database, not a per-node copy.

10. Load Balancing with HAProxy (Single Endpoint for All Nodes)

Every command so far has pointed clients at one node's IP directly. HAProxy gives every client — pgAdmin, DBeaver, JDBC/psycopg applications, and browsers hitting the web UI — a single virtual endpoint that stays up even if any one node goes down, and spreads connections across all three automatically. This section fronts the ports already opened and working in Sections 3 and 8 (YSQL 5433, YCQL 9042, and the nginx-proxied, password-protected web UIs 7080/9080) rather than exposing the native admin ports (7000/9000) directly.

10.1 Where HAProxy runs

Ideally a separate small VM, so the load balancer isn't also one of the three database nodes — if that node goes down, you lose a DB node and your single entry point at the same time. This guide uses 192.168.87.90 as the HAProxy host; substitute your own IP throughout.

10.2 Install HAProxy

```
sudo yum install -y haproxy
```

10.3 Configure /etc/haproxy/haproxy.cfg

```
sudo cp /etc/haproxy/haproxy.cfg /etc/haproxy/haproxy.cfg.orig

sudo tee /etc/haproxy/haproxy.cfg <<'EOF'
global
    log          127.0.0.1 local2
    maxconn      4096
    daemon

defaults
    log          global
    mode         tcp
    option       tcplog
    timeout connect 5s
    timeout client 30m
    timeout server 30m
    retries      3

# ----- YSQL (5433) -----
frontend ft_ysql
    bind *:5433
    default_backend bk_ysql

backend bk_ysql
    balance roundrobin
    option tcp-check
    server node1 192.168.87.133:5433 check
    server node2 192.168.87.194:5433 check
    server node3 192.168.87.195:5433 check

# ----- YCQL (9042) -----
frontend ft_ycql
```

```

bind *:9042
default_backend bk_ycql

backend bk_ycql
    balance roundrobin
    option tcp-check
    server node1 192.168.87.133:9042 check
    server node2 192.168.87.194:9042 check
    server node3 192.168.87.195:9042 check

# ----- Master Web UI (7080, nginx+auth) -----
frontend ft_master_ui
    bind *:7080
    default_backend bk_master_ui

backend bk_master_ui
    balance roundrobin
    option tcp-check
    server node1 192.168.87.133:7080 check
    server node2 192.168.87.194:7080 check
    server node3 192.168.87.195:7080 check

# ----- TServer Web UI (9080, nginx+auth) -----
frontend ft_tserver_ui
    bind *:9080
    default_backend bk_tserver_ui

backend bk_tserver_ui
    balance roundrobin
    option tcp-check
    server node1 192.168.87.133:9080 check
    server node2 192.168.87.194:9080 check
    server node3 192.168.87.195:9080 check

# ----- HAProxy stats page -----
listen stats
    bind *:8404
    mode http
    stats enable
    stats uri /
    stats refresh 10s
    stats auth admin:ChangeThisPassword
EOF

```

`option tcp-check` confirms the port accepts a TCP connection — sufficient here since 7080/9080 already sit behind nginx Basic Auth (Section 8) and HAProxy doesn't need to authenticate to prove the backend is alive.

10.4 SELinux — allow HAProxy to connect out to arbitrary ports

```
sudo setsebool -P haproxy_connect_any 1
```

10.5 Firewall on the HAProxy host

```

sudo firewall-cmd --permanent --zone=public --add-port=5433/tcp
sudo firewall-cmd --permanent --zone=public --add-port=9042/tcp
sudo firewall-cmd --permanent --zone=public --add-port=7080/tcp

```

```
sudo firewall-cmd --permanent --zone=public --add-port=9080/tcp
sudo firewall-cmd --permanent --zone=public --add-port=8404/tcp # stats page, optional
sudo firewall-cmd --reload
```

10.6 Enable, start, and verify

```
sudo systemctl enable --now haproxy
sudo systemctl status haproxy --no-pager
```

```
# YSQL through the LB:
ysqlsh -h 192.168.87.90 -U yugabyte -W
```

```
# YCQL through the LB:
ycqlsh 192.168.87.90 9042 -u cassandra -p 'yourpassword'
```

```
# Web UIs through the LB:
curl -u admin:YourPassword -o /dev/null -s -w "%{http_code}\n" http://192.168.87.90:7080/
curl -u admin:YourPassword -o /dev/null -s -w "%{http_code}\n" http://192.168.87.90:9080/
```

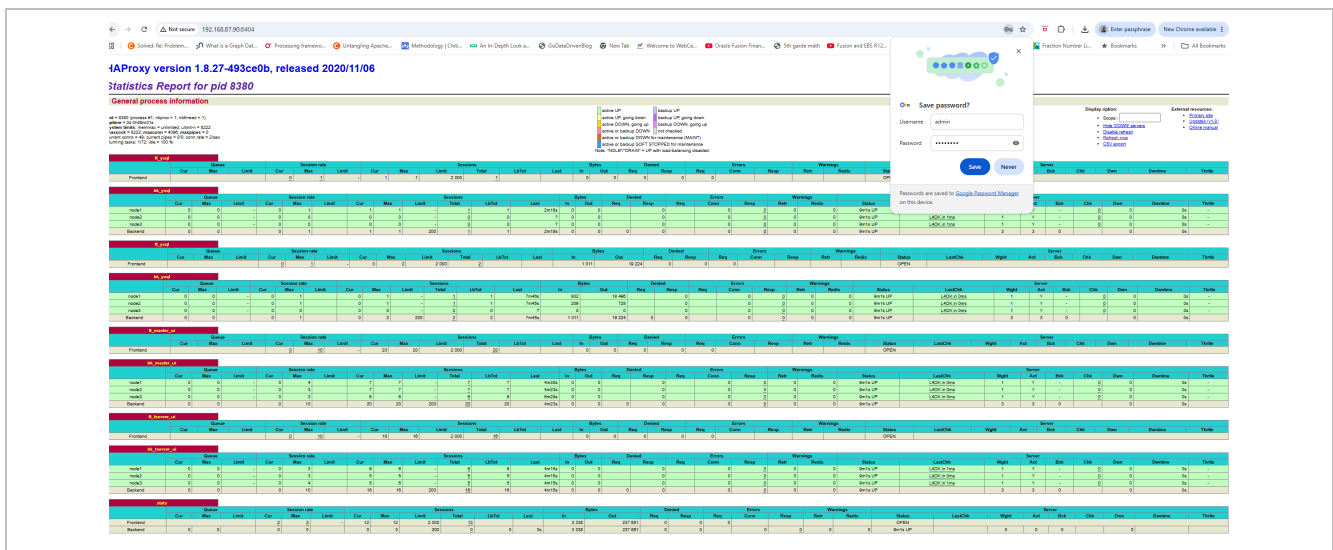
10.7 What it looks like end to end

Browsing to `http://192.168.87.90:7080` goes through HAProxy to whichever node it routes the request to — the dashboard itself is identical regardless of node, since all three report the same cluster-wide state:

The screenshot displays the YugabyteDB Master UI. The 'Overview' section provides a summary of the cluster's state, including the universe UUID, replication factor, number of nodes and user tables, and the status of cluster balance and load. The 'Masters' section lists the three nodes in the cluster, their roles, and their last update times. A password management dialog is visible, indicating that the user is configuring or verifying access credentials.

Master UI reached through the HAProxy endpoint (192.168.87.90:7080) — behind the scenes this request was routed to node2, but the page is indistinguishable from browsing a node directly.

The HAProxy stats dashboard (192.168.87.90:8404) confirms all four backends — YSQL, YCQL, Master UI, and TServer UI — see all three nodes UP and receiving traffic:



HAProxy stats page showing `bk_ysql`, `bk_ycql`, `bk_master_ui`, and `bk_tserver_ui` backends, each with `node1/node2/node3` reporting UP (green) and session counts spread across all three — proof the load balancer is actively distributing traffic, not just passing it to one node.

Test real failover, not just the config: Stop `yb-tserver` on one node and refresh the HAProxy stats page — that node should flip red within a couple of health-check intervals while HAProxy keeps routing to the other two. Confirming the config loaded is not the same as confirming failover actually works under a real outage.

TCP passthrough, not sticky sessions: HAProxy is proxying raw TCP here, so a given browser tab's requests can land on a different backend node between page loads. This is harmless for the web UI (every node reports the same cluster-wide state) and is exactly what YSQL/YCQL clients want (spread load across nodes) — but it's worth knowing so a UI page that appears to "switch nodes" on refresh isn't mistaken for a bug.

10.8 Connecting pgAdmin Through the Load Balancer

pgAdmin connects to the HAProxy endpoint exactly like connecting to any single PostgreSQL server — the same Section 7.4 settings apply, just pointed at the load balancer's IP instead of one node.

Step 1 — Register → Server → Connection tab

Enter the load balancer's IP as Host, port 5433, Maintenance database yugabyte, and the YSQL username/password. Leave Host/Port pointed at the LB — not at any individual node — so pgAdmin benefits from HAProxy's health checking and traffic distribution.

The screenshot shows the 'Register - Server' dialog box with the 'Connection' tab selected. The fields are filled as follows:

Field	Value
Host name/address	192.168.87.90
Port	5433
Maintenance database	yugabyte
Username	yugabyte
Kerberos authentication?	☐
Password	
Save password?	☒
Role	

Register Server → Connection tab filled in: Host 192.168.87.90 (the HAProxy endpoint), Port 5433, Maintenance database yugabyte, Username yugabyte.

Leave "Save password?" off on first connect: Toggling Save password on can trigger a one-time pgAdmin master-password/keyring prompt. If that dialog opens behind the main window and isn't noticed quickly, pgAdmin pauses before sending its SASL/SCRAM password response — and since the TCP connection to the backend is already open and idle, YugabyteDB's own 60-second authentication_timeout eventually kills it server-side. This surfaces in pgAdmin as "server closed the connection unexpectedly," which looks like a network or load-balancer problem but is actually a stalled client-side dialog. Connect once with Save password off to confirm credentials work, then enable it afterward if wanted.

Step 2 — Verify the connection

Once connected, the pgAdmin tree shows the server as a live, expandable node with yugabyte (and any other databases, e.g. northwind from Section 9) listed underneath — the same Postgres-style browsing experience as any other server, just transparently load-balanced across all three nodes underneath.

192.168.87.90

General Connection Parameters SSH Tunnel Advanced

Host name/address 192.168.87.90

Port 5433

Maintenance database yugabyte

Username yugabyte

Kerberos authentication? ☐

Role

Service

Close Reset Save

pgAdmin successfully connected through the HAProxy endpoint (192.168.87.90) — the properties/connection view with no error banner confirms the session is live.

Quick connectivity check (before touching pgAdmin)

If a connection ever fails and it's unclear whether the problem is network-level or application-level, check raw TCP reachability first from the client machine — this isolates "can't reach the port at all" from "reached it, but the database/application layer is having trouble":

```
# PowerShell (Windows client machine):
Test-NetConnection 192.168.87.90 -Port 5433
# TcpTestSucceeded : True → port is reachable; look at the application/auth layer next
# TcpTestSucceeded : False → firewall/routing issue between this machine and the LB
```

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\mukes> Test-NetConnection 192.168.87.90 -Port 5433

ComputerName      : 192.168.87.90
RemoteAddress     : 192.168.87.90
RemotePort        : 5433
InterfaceAlias    : Wi-Fi
SourceAddress     : 192.168.87.110
TcpTestSucceeded  : True

PS C:\Users\mukes> |
```

Test-NetConnection confirming raw TCP reachability to the load balancer on port 5433 — this succeeding while pgAdmin still failed was the key clue that pointed at an application-layer stall rather than a network block.

Diagnostic order that actually finds the answer: When a client can't connect: (1) raw TCP test (Test-NetConnection / telnet) to rule networking in or out, (2) connect directly to one node, bypassing the LB, to

isolate HAProxy from the backend, (3) tail the HAProxy log (Section 10 setup) during a live attempt to see which node it routed to and how many bytes were exchanged, (4) tail that specific node's YSQL log (`/opt/data0/yb-data/yb-data/tserver/logs/postgresql-*.log`) for the same timestamp. In this guide's own troubleshooting, that sequence showed the LB and backend were both healthy — ysqlsh authenticated through the LB with no issue — and narrowed the fault down to a stalled client-side dialog in pgAdmin, not the infrastructure.

11. Remaining Hardening for Production

This guide brings the cluster to a secure, working baseline: authenticated YSQL/YCQL, password-protected web UIs behind nginx, and an explicit port allow/deny policy. Before running production workloads, address the following:

11.1 Encrypt traffic (TLS)

- Node-to-node and client-to-server TLS for YSQL/YCQL (`--use_node_to_node_encryption`, `--use_client_to_server_encryption`, `--certs_dir` flags on `yb-master/yb-tserver`) — currently all database traffic is unencrypted on the wire.
- TLS on the nginx reverse proxy (`listen 7080 ssl; + ssl_certificate/ssl_certificate_key`) — currently Basic auth credentials for the web UI travel unencrypted, as flagged in Section 8.6.

11.2 SELinux and firewall posture

- If any node was left in Permissive mode during setup, return it to Enforcing (`sudo setenforce 1; sudo sed -i 's/^SELINUX=.*/SELINUX=enforcing/' /etc/selinux/config`) and re-verify nginx/YugabyteDB still start cleanly.
- Re-run the verification in Section 3.6 and 8.6's "verify the block" callout periodically, especially after any `firewalld` or SELinux policy change.

11.3 Backups and disaster recovery

- Configure `yb-admin create_snapshot` / distributed backups to cloud storage (S3/GCS/Azure) or NFS on a schedule.
- Document and test a node-failure runbook: stop `yb-tserver/yb-master` on one node deliberately and confirm the cluster keeps serving reads/writes (replication factor 3 tolerates 1 node down).

11.4 Operational hygiene

- Rotate the `yugabyte/cassandra/admin` passwords set in Sections 5.4 and 8.2 on a regular cadence, and store them in a secrets manager rather than shell history.
- Set up monitoring/alerting (Prometheus scrape of the metrics endpoints noted in Section 3.4, plus Grafana dashboards) rather than relying on manual `curl` checks.
- Plan capacity for a 4th+ node and test `yb-admin add/remove` of a tablet server before you need to do it under pressure.
- Keep this guide's port table (Section 1.1) in sync with any future flag changes — a new `--metrics` or `--webserver` port introduced later needs the same allow/deny treatment as the ones covered here.

This guide's scope: Sections 1–9 above are sufficient for a secure, working 3-node development/test cluster with authenticated access and a locked-down port surface. Section 10 items are recommended before production traffic, but are outside this guide's scope and depend on your organization's TLS/PKI, backup, and monitoring standards.